

Unit 3.3 Interprocess Communication

- How one process can pass the information to the another? – How to make sure two or more processes do not get into each other 's way when engaging in critical activities?
- How to maintain the proper sequence when dependencies are presents?

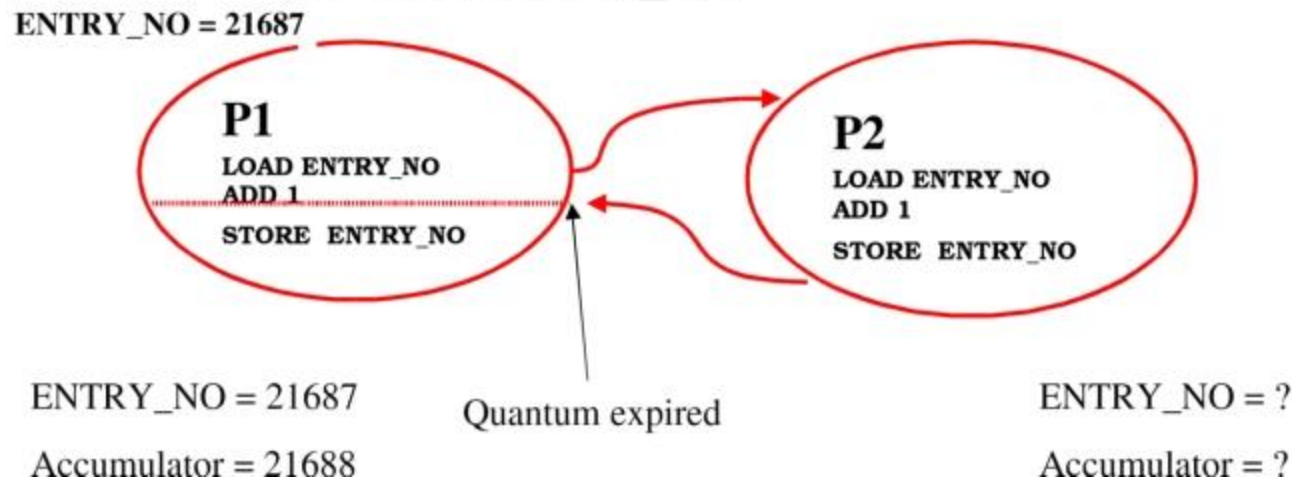
Thus, InterProcess Communication (IPC) and synchronization

IPC provide the mechanism to allow the processes to communicate and to synchronize their actions.

Race Conditions

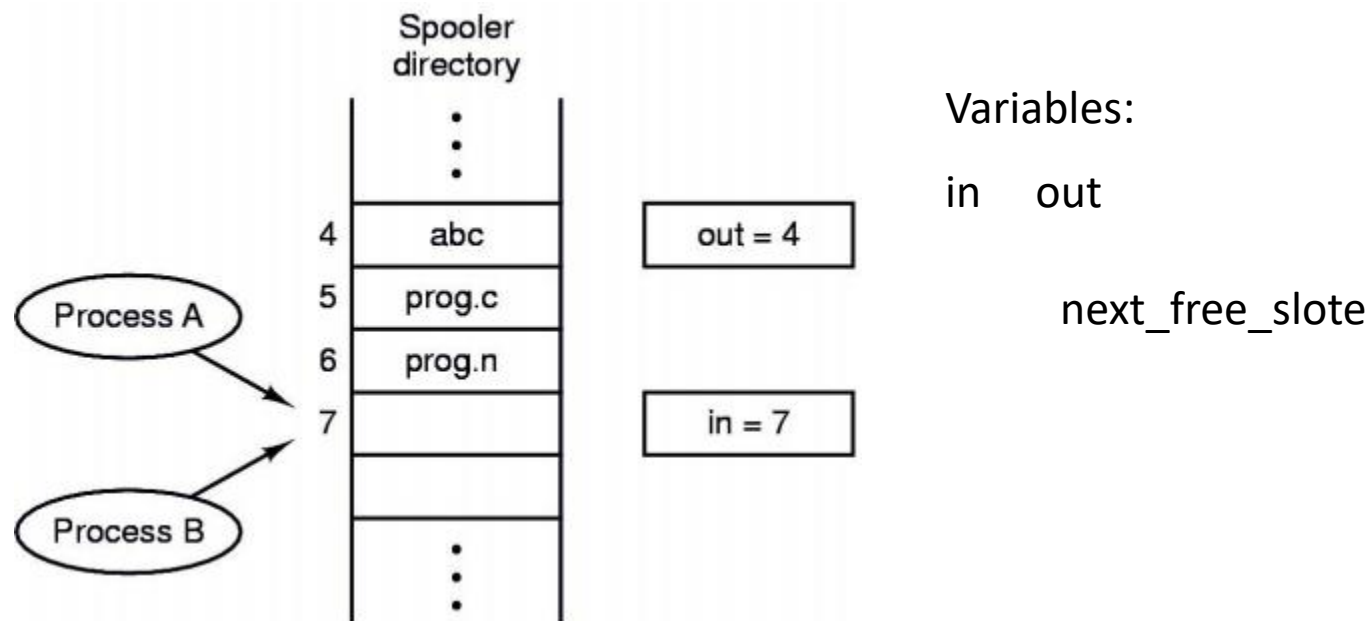
Scenario1: Suppose two user working in a computer, there is a mechanism to monitor continuously the total number of lines each users have entered, each user entry is recorded globally, different process monitoring different users .

Each process : LOAD ENTRY_NO ADD 1
STORE ENTRY_NO



Race Conditions

Scenario2: a print spooler: when a process wants to print a file, it enters the file name in a special spooler directory. Another process, print daemon, periodically checks to see if there are any files to be printed, print them and removes their names from the directory.



Situation where two or more processes are reading or writing some shared data and the final result depends on who runs precisely, are called race conditions

Mutual Exclusion

Possibilities of race:

- many concurrent process read the same data.
- one process reading and another process writing same data.
- two or more process writing same data.

Solution: *prohibiting more than one process from reading writing the same data at the same time - **Mutual Exclusion** .*

Mutual Exclusion:

Some way of making sure that if one process is using a shared variables or files, the other process will be excluded from doing the same thing. It prevents the race condition to occur.

Mutual Exclusion techniques

- Interrupt Disabling
- Lock Variables
- Strict Alternation
- Peterson's Algorithm
- Hardware Solution - TSL

Critical Section

Problem: How to avoid race?

Fact: The part of the time, process is busy doing internal computations and other things that do not lead to the race condition.

Code executed by the process can be grouped into sections, some of which require access to shared resources, and other that do not. The section of the code that require access to shared resources are called critical section .

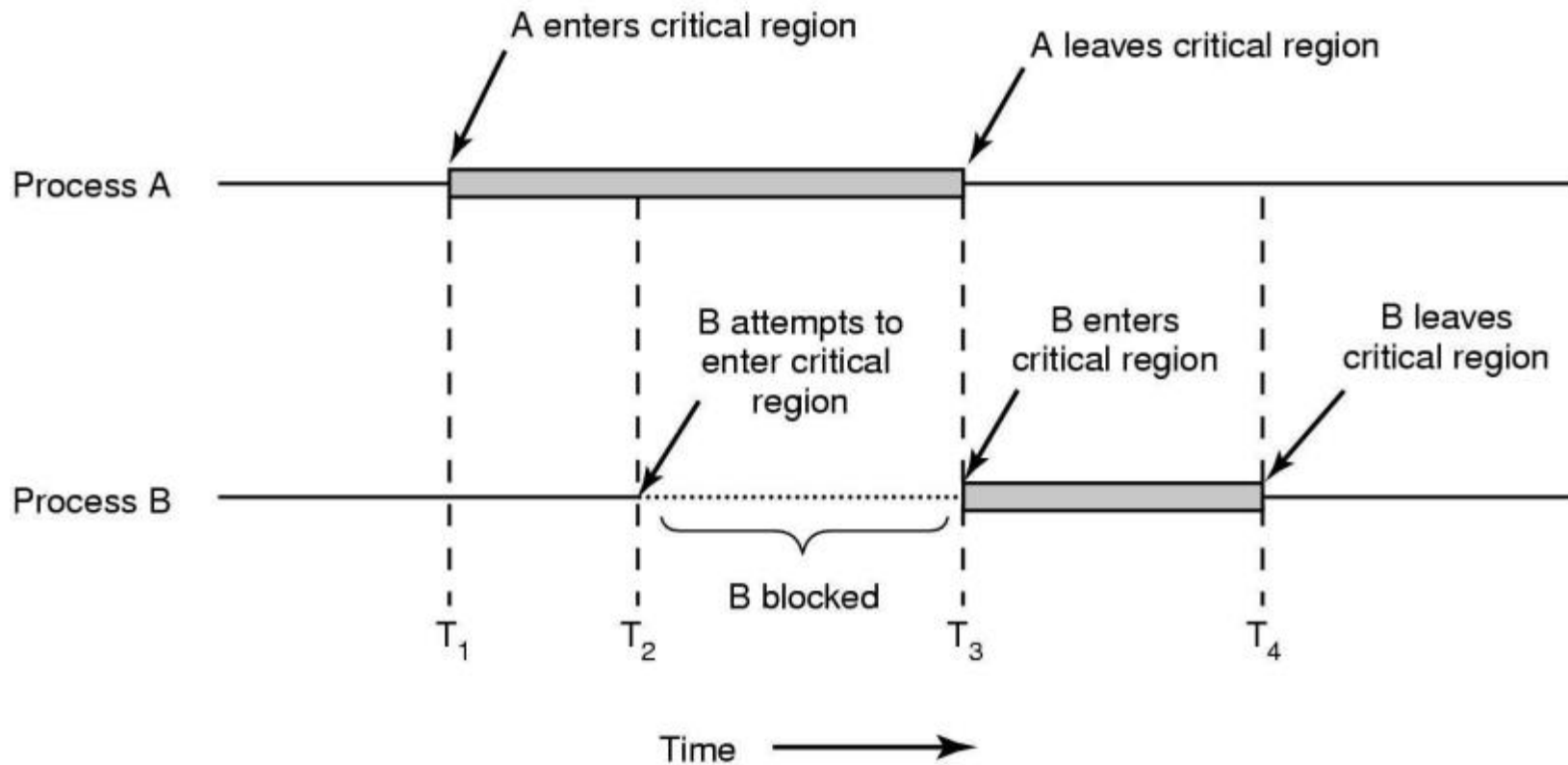
Critical Section

- When a process is accessing a shared modifiable data, the process is said to be in critical section.
- All other processes (those access the same data) are excluded from their own critical region.
- All other processes may continue executing outside their CR.
- When a process leaves its critical section, then another processes waiting to enter its own CR should be allowed to proceed.

CR must satisfy the following conditions:

1. No two processes may be simultaneously inside their CRs (mutual exclusion).
2. No assumptions may be made about the speeds or number of CPUs.
3. No process running outside its CR may block other process.
4. No process should have to wait forever to enter its CR.

Critical Section



Mutual exclusion using critical regions

ME with Busy Waiting

Interrupt Disabling

Each process disable all interrupts just after entering its CR and re-enable them just before leaving it.

No clock interrupt, no other interrupt, no CPU switching to other process until the process turn on the interrupt.

DisableInterrupt() // perform CR task

EnableInterrupt()

Advantages:

Mutual exclusion can be achieved by implementing OS primitives to disable and enable interrupt.

Problems:

- allow the power of interrupt handling to the user.
- The chances of never turned on – is a disaster.
- it only works in single processor environment.

ME with Busy Waiting

Lock Variables

A single, shared (lock) variable, initially 0. When a process wants to enter its CR, it first test the lock. If the lock is 0, the process set it to 1 and enters the CR. If the lock is already 1, the process just waits until it becomes 0. Lock=0;

```
If(lock==0)
```

```
{
```

```
    lock=1
```

```
    Enter in CR and operate
```

```
    lock=0
```

```
}
```

Advantages: seems no problems.

Problems: problem like spooler directory; suppose that one process reads the lock and sees that it is 0, before it can set lock to 1, another process scheduled, enter the CR, set lock to 1 and can have two process at CR (violates mutual exclusion).

ME with Busy Waiting

Strict Alternation

*Processes share a common integer variable turn.
If $turn == i$ then process P_i is allowed to execute in its CR,
if $turn == j$ then process P_j is allowed to execute.*

Advantages : Ensures that only one process at a time can be in its CR.

Problems: strict alternation of processes in the execution of the CR. What happens if process i just finished CR and again need to enter CR and the process j is still busy at non-CR work? (violate condition 3)

ME with Busy Waiting

Peterson's Algorithm

- Before entering its CR, each process call *enter_region()* with its own process number, 0 or 1 as parameter.
- Call will cause to wait, if need be, until it is safe to enter.
- When leaving CR, the process calls *leave_region()* to indicate that it is done and to allow other process to enter CR.

Advantages: Preserves all conditions.

Problems: difficult to program for n-processes system and less efficient.

ME with Busy Waiting

Hardware Solution - TSL

help from the hardware

- Test and Set Lock (TSL) instruction reads the contents of the memory word lock (shared variable) into the register and then stores nonzero value at the memory address lock.
- This automatically as in a un-interruptible time unit.
- The CPU executing TSL locks the memory bus to prohibit other CPUs from accessing memory until it is done.
- When lock is 0, any process may set it to 1 using the TSL instruction.
- When it is done the process lock back to 0.

ME with Busy Waiting

Hardware Solution - TSL

enter_region:

*TSL register, lock |copy lock to register and set lock to 1 CMP register,
#0 |was lock 0?*

*JNE enter_region |if it was non zero, lock was set, so loop RET
|return to caller; critical_region();*

leave_region:

MOVE lock, #0 |store a 0 in lock

RET |return to caller noncritical_section();

Advantages: *Preserves all condition, easier programming task and
improve system efficiency.*

Problems: *difficulty in hardware design.*

Busy Waiting Alternate?

Busy waiting:

When a process want to enter its CR, it checks to see if the entry is allowed, if it is not, the process just sits in a tight loop waiting until it is.

Waste of CPU time for NOTHING!

Possibility of *Sleep* and *Wakeup* pair instead of waiting.

Sleep causes to process to block, until another process wakes it up.

Sleep and Wakeup

Producer-Consumer Problem

- Two process share a common, fixed sized buffer.
- Suppose one process, producer, is generating information that the second process, consumer, is using.
- Their speed may be mismatched, if producer insert item rapidly, the buffer will full and go to the sleep until consumer consumes some item, while consumer consumes rapidly, the buffer will empty and go to sleep until producer put something in the buffer.

Producer and Consumer

```
#define N = 100                                /* number of slots in the buffer*/
int count = 0;                                /* number of item in the buffer*/
void producer(void)
{
    int item;
    while(TRUE)
    {
        /* repeat forever*/
        if (count == N)
            sleep();                          /* if buffer is full go to sleep*/
        if (count == 1)
            wakeup(consumer);
        item = produce_item();                /*generate next item*/
        insert_item(item);                    /* put item in buffer */
        count = count +1;                     /*increment count */
    }
}
```

Producer and Consumer

```
void consumer(void)
{
    int item;
    while(TRUE)
    {
        /* repeat forever*/
        if(count == 0)
            sleep();          /* if buffer is empty go to sleep */
        if (count == N-1)
            wakeup(producer);
        item = remove_item(); /*take item out
        count = count-1;      /*decrement count*/
        consume_item();
    }
}
```

Sleep and Wakeup

Producer-Consumer Problem

Problem:

-leads to race as in spooler directory.

-What happen if

When the buffer is empty, the consumer just reads count and quantum is expired, the producer inserts an item in the buffer, increments count and wake up consumer. The consumer not yet asleep, so the wakeup signal is lost, the consumer has the count value 0 from the last read so go to the sleep. Producer keeps on producing and fill the buffer and go to sleep, both will sleep forever.

Think: If we were able to save the wakeup signal that was lost.....